Smart Resume Screening and Interview Confirmation

Smart Resume Screening is a recruiter-focused application that turns resume screening into an explainable, evidence-based workflow.

The project helps a recruiter:

? Create and manage multiple job openings.

? Upload multiple candidate resumes for a selected opening.

? Extract structured information from PDF, DOCX, and TXT resumes.

? Compare each resume with the job description.

? Verify publicly available proof of work from GitHub and LeetCode.

? Rank candidates using an explainable score.

? Review the evidence behind every recommendation.

? Shortlist, reject, interview, or delete candidates.

? Send a secure interview-availability confirmation link.

? See whether the candidate accepted or declined before scheduling the interview.

The application is designed to support human decision-making rather than replace the recruiter. The system recommends and explains; the recruiter makes the final decision.

Project motive

Traditional resume screening is often:

? Time-consuming when many resumes are received.

? Dependent on keyword matching rather than actual relevance.

? Difficult to audit because the reason for a ranking is unclear.

? Vulnerable to exaggerated technical claims.

? Disconnected from the interview-confirmation process.

This project addresses those problems by combining resume parsing, semantic matching, proof-of-work verification, explainable scoring, blind screening, recruiter feedback, and candidate confirmation in one workflow.

Main goals

? **Reduce manual screening time** without hiding the reason for a decision.

? **Improve technical-skill confidence** by checking public work rather than trusting claims alone.

? **Keep recruiters in control** through shortlist/reject actions and human approval.

? **Avoid unnecessary interview scheduling** by asking a shortlisted candidate to confirm availability first.

? **Preserve candidate privacy** through recruiter ownership checks, blind review, expiring links, and candidate deletion.

High-level architecture

```
Recruiter browser
    |
    |  frontend/index.html
    |  frontend/confirmation.html
    |
    v
FastAPI backend
    |
    +-- Authentication and JWT security
    +-- Job and candidate APIs
    +-- Resume extraction and NLP parsing
    +-- Semantic job matching
    +-- GitHub and LeetCode verification
    +-- Explainable scoring
    +-- Interview confirmation workflow
    |
    v
SQLAlchemy database
    |
    +-- Users
    +-- Jobs
    +-- Candidates
    +-- Feedback
    +-- WhatsApp confirmations
```

Important directories

Path	Responsibility
`frontend/index.html`	Main recruiter dashboard, job management, uploads, ranking, candidate drawer, actions, and loading states
`frontend/confirmation.html`	Candidate-facing interview-availability confirmation page
`backend/app/main.py`	FastAPI application, router registration, startup schema setup, health endpoint
`backend/app/models.py`	SQLAlchemy database models
`backend/app/schemas.py`	Request validation models
`backend/app/database.py`	Database engine, session, and declarative base
`backend/app/security.py`	Password hashing, JWT creation, and authenticated-user dependency
`backend/app/routers/auth.py`	Registration and login
`backend/app/routers/jobs.py`	Job openings, deletion, candidate deletion, and weight management
`backend/app/routers/candidates.py`	Resume upload, ranking, candidate details, actions, PDF viewing, and confirmation-link creation
`backend/app/routers/whatsapp.py`	Public candidate confirmation details and accept/decline endpoints
`backend/app/services/extraction.py`	PDF, DOCX, TXT, and OCR extraction
`backend/app/services/nlp.py`	Resume parsing and structured field extraction
`backend/app/services/matching.py`	Semantic job-description matching
`backend/app/services/proof_of_work.py`	GitHub and LeetCode evidence verification
`backend/app/services/scoring.py`	Weighted, explainable ranking and weight learning
`backend/app/services/pipeline.py`	Coordinates processing, matching, verification, and scoring
`backend/app/services/whatsapp.py`	Secure confirmation-token generation and expiry
`backend/tests/test_pipeline.py`	API and workflow regression tests

Technology stack

? Python

? FastAPI

? SQLAlchemy

? SQLite by default

? PostgreSQL support through `DATABASE_URL`

? Pydantic

? JWT bearer authentication

? `pdfplumber` for PDF text extraction

? `python-docx` for DOCX extraction

? Optional Tesseract/PDF-to-image OCR for scanned resumes

? Optional Sentence Transformers/SBERT for semantic matching

? TF-IDF fallback when SBERT is unavailable

? GitHub REST API

? LeetCode GraphQL endpoint

? Vanilla HTML, CSS, and JavaScript frontend

Optional packages are detected at runtime where supported. The application falls back to simpler local behavior when an optional dependency or external verification service is unavailable.

Running the application

Backend

From the project root:

```
cd backend
python -m venv .venv
```

Activate the environment:

```
# Windows PowerShell
.\.venv\Scripts\Activate.ps1

# macOS/Linux
source .venv/bin/activate
```

Install dependencies:

```
pip install -r requirements.txt
```

Create environment configuration:

```
# Windows PowerShell
Copy-Item .env.example .env

# macOS/Linux
cp .env.example .env
```

Start FastAPI:

```
uvicorn app.main:app --reload
```

The backend is normally available at:

? API: `http://127.0.0.1:8000`

? Swagger UI: `http://127.0.0.1:8000/docs`

? OpenAPI JSON: `http://127.0.0.1:8000/openapi.json`

? Health: `http://127.0.0.1:8000/health`

Frontend

The frontend is intentionally simple and does not require a JavaScript build system. Serve the `frontend` directory with a static server.

For example:

```
cd frontend
python -m http.server 5500
```

Open:

```
http://127.0.0.1:5500/index.html
```

The recruiter frontend communicates with the FastAPI backend on port `8000`. If the backend runs on another host or port, update the frontend API-origin configuration.

Tests

From the project root:

```
python -m pytest
```

The test suite covers authentication, multiple job openings, resume processing, ranking, proof-of-work behavior, demo-number persistence, confirmation responses, ownership protection, and resume PDF delivery.

Configuration

Important environment variables are documented in `backend/.env.example`.

Common settings include:

Variable	Purpose
`DATABASE_URL`	Database connection URL; SQLite is the default
`JWT_SECRET`	Secret used to sign authentication tokens
`GITHUB_TOKEN`	Optional GitHub token for higher API limits
`ENABLE_EXTERNAL_VERIFICATION`	Enables/disables GitHub and LeetCode checks
`WHATSAPP_PUBLIC_URL`	Public base URL used when generating externally shareable confirmation links
Upload and processing limits	Restrict file size and number of resumes per request

Use a strong, private `JWT_SECRET` in any non-development environment. Never commit `.env` or API tokens.

Complete workflow

1. Recruiter registration and login

The recruiter creates an account or logs in. The backend returns a JWT bearer token. All recruiter-only endpoints require:

```
Authorization: Bearer <access_token>
```

The frontend stores the authenticated session and restores the recruiter workspace, selected job, and selected candidate where possible.

2. Create a job opening

The recruiter enters:

? Job title

? Job description

? Required skills, or lets the backend extract skills from the description

? Minimum experience

? Education level

? Whether proof of work is required

The system supports multiple openings. The frontend can create another opening without leaving the current workflow, and the recruiter can select one opening to view its candidates.

3. Upload resumes

The recruiter uploads one or more PDF, DOCX, or TXT files to a selected job.

For each resume, the backend:

? Reads and validates the file.

? Checks the upload-size limit.

? Extracts text and hyperlinks.

? Runs OCR when configured and needed for scanned PDFs.

? Detects hidden or suspicious text.

? Extracts name, email, phone, sections, skills, experience, education, GitHub, and LeetCode links.

? Creates a content hash to prevent duplicate uploads for the same job.

? Stores the derived screening data.

? Stores original PDF bytes for the recruiter resume viewer.

? Assigns a configured demo WhatsApp number for the confirmation demonstration.

? Runs matching, proof-of-work checks, and scoring.

The frontend keeps the loading state active until the complete upload and ranking request finishes. A finished ranking request does not navigate away from the selected opening.

4. Ranking and explanation

Candidates are ranked by final score. Each candidate exposes:

? Final score and fit tier

? Matched skills

? Missing skills

? Component scores

? Component contributions

? Proof-of-work evidence

? Integrity flags

? Plain-English reasons

? Manual-review requirement

? Recruiter feedback status

? Interview-confirmation status

The recruiter can also use blind mode. In blind mode, identity fields are hidden from list and detail responses where appropriate, while semantic matching uses text with identifying information removed.

5. Recruiter decision

The recruiter can:

? Shortlist

? Reject

? Mark for interview

? Add a note

? Reverify proof of work

? Open the original PDF when it was uploaded as a PDF

? Delete one candidate

? Delete all candidates for the selected job

? Export the ranking as CSV

The system does not automatically reject candidates. The recruiter remains responsible for the final decision.

6. Interview availability confirmation

After a candidate is shortlisted, the recruiter generates a secure confirmation link.

The recruiter can share the link manually through WhatsApp, SMS, or email. The free workflow does not require the WhatsApp Cloud API and does not send messages automatically.

The candidate opens `frontend/confirmation.html`, which:

? Reads the secure token from the URL.

? Requests the confirmation details from the backend.

? Displays the candidate-facing job information.

? Shows the job title, description, required skills, experience requirement, and education requirement.

? Lets the candidate choose **Accept** or **Decline**.

? Sends the decision to the backend.

The recruiter dashboard then displays:

? Awaiting response

? Interview accepted

? Interview declined

? Response timestamp

? Confirmation expiry

This creates a confirmation checkpoint before an interview is scheduled.

Unique implementation: confirmation before interview

The confirmation workflow is more than a button that opens WhatsApp. It is a secure, trackable state machine.

Link generation

The backend:

? Confirms that the recruiter owns the candidate.

? Confirms that the candidate is shortlisted.

? Generates a random token.

? Stores only a SHA-256 hash of the token.

? Stores an expiry time.

? Stores the last four digits of the allocated demonstration number.

? Marks the request as `link_generated`.

? Returns a link for the recruiter to share.

The raw token is not stored in the database. A person with database access cannot directly use the stored hash as the confirmation link.

Candidate decision

The candidate-facing endpoints are public by design, but access requires the unguessable token. The backend:

? Rejects invalid tokens.

? Rejects expired links.

? Rejects decisions after a previous decision has been recorded.

? Accepts only `accepted` or `declined`.

? Stores the final status.

? Stores `responded_at`.

Why this matters

The recruiter can distinguish:

? A candidate who was never contacted.

? A candidate who received a link but has not replied.

? A candidate who accepted availability.

? A candidate who declined availability.

? A candidate whose link expired.

This prevents the recruiter from treating every shortlisted candidate as interview-confirmed.

Proof-of-work verification

Resume claims are not treated as automatically true. For technical candidates, the system checks public evidence where links are available.

GitHub checks

The GitHub verification service can inspect public profile and repository information, including:

? Username and profile availability

? Owned repositories

? Non-fork repositories

? Commits

? Recent activity

? Commits during the last 90 days

? Estimated active days over 30, 90, and 365 days

? Active repositories during the last 12 months

? Stars and forks

? Technologies detected in repositories

? Repository ownership or contributor relationship

? Whether a claimed skill is supported by the candidate's own work

The proof-of-work evidence shown in the recruiter UI can include values such as:

```
10 own repos
0 commits in 90 days
0 active days in 90 days
2 stars
HTML, CSS, JavaScript
```

These values are evidence, not a guarantee of ability. Public API limits and the visibility of repositories affect the result.

LeetCode checks

When a LeetCode username is found, the system can request public GraphQL data such as:

? Total solved problems

? Easy, medium, and hard solved counts

? Contest rating

? Ranking-related information

? Contest participation where available

LeetCode does not provide a stable official public API for all of these values. If the service is unavailable or changes its response format, the candidate is not silently treated as having no ability; the result is flagged as unverifiable or unavailable.

Claim verification

The service compares claimed skills with public technical evidence:

? A claimed programming skill may be checked against repository language or technology evidence.

? A claimed GitHub project is checked against ownership or contributor evidence.

? A data-structures-and-algorithms claim may be compared with LeetCode solve evidence.

Claims are categorized as verified or unverified where the available evidence supports that distinction.

Important limitations

Proof of work cannot fully prove:

? That a public account belongs to the candidate.

? The quality of every line of code.

? Private repositories or private contributions.

? That all public commits were authored independently.

? That a candidate is currently available for an interview.

Recruiters should use proof-of-work as supporting evidence and should disclose public-profile verification where required.

Explainable scoring

The default score uses these components:

Component	Default weight
Skills match	30%

Component	Default weight
Experience	15%
Education	10%
Semantic job-description match	25%
Proof of work	20%

The score is calculated as a weighted sum of active components:

```
final score = weighted component score × integrity multiplier
```

Component behavior

? **Skills:** proportion of required skills found in the parsed resume.

? **Experience:** candidate experience compared with the job minimum.

? **Education:** education level compared with the job requirement.

? **Semantic match:** similarity between resume content and the job description.

? **Proof of work:** evidence from GitHub, LeetCode, and claim verification.

If proof of work is not applicable or cannot be verified because an external service is unavailable, that component is excluded and the remaining weights are renormalized. A temporary API outage should not silently penalize every candidate.

Integrity signals

The safety layer can flag:

? Hidden, tiny, or white PDF text.

? Zero-width characters.

? Instruction-like text intended to manipulate the screening process.

? Excessive keyword repetition or keyword stuffing.

Integrity flags can reduce the score and require manual review. The scorer returns the reason so the recruiter can inspect it.

Fit tiers

Score	Tier
75–100	`strong_fit`
50–74.99	`moderate_fit`
Below 50	`low_fit`

Feedback-based weight learning

The recruiter can submit shortlist and reject decisions. After enough decided candidates, the backend can use those decisions to suggest new weights through logistic regression.

The suggestion is not applied by default. The recruiter must explicitly request `apply=true`, keeping the human in control.

Endpoint reference

Unless stated otherwise, endpoints require:

```
Authorization: Bearer <access_token>
```

Health and metadata

`GET /health`

Public health check.

Returns:

- ? Application status
- ? Semantic backend in use
- ? Whether external verification is enabled
- ? Whether a GitHub token is configured

Authentication

`POST /auth/register`

Creates a recruiter account.

Example request:

```
{
  "email": "recruiter@example.com",
  "password": "strong-password",
  "company": "Example Company"
}
```

Returns the user ID, email, and bearer token.

`POST /auth/login`

Authenticates a recruiter and returns a bearer token.

Example request:

```
{
  "email": "recruiter@example.com",
  "password": "strong-password"
}
```

Job openings

`POST /jobs`

Creates a job opening. If `required_skills` is omitted, skills are extracted from the job description.

Supported fields include:

- ? `title`
- ? `description`
- ? `required_skills`
- ? `min_experience`
- ? `education_level`
- ? `requires_proof_of_work`

`GET /jobs`

Lists the authenticated recruiter's job openings, newest first.

`GET /jobs/{job_id}`

Returns one owned job opening, including candidate count and normalized weights.

`DELETE /jobs/{job_id}`

Deletes an owned job opening and its related candidates according to the database relationships.

`DELETE /jobs/{job_id}/candidates`

Deletes all candidates belonging to one owned job opening.

Returns the number of deleted candidates.

`PUT /jobs/{job_id}/weights`

Updates scoring weights and rescored all candidates for that job.

Example:

```
{
  "skills": 0.30,
  "experience": 0.15,
  "education": 0.10,
  "semantic": 0.25,
  "proof_of_work": 0.20
}
```

Weights are normalized before use.

`POST /jobs/{job_id}/weights/learn?apply=false`

Learns a suggested weighting from recruiter decisions.

? `apply=false`: return a suggestion only.

? `apply=true`: save the suggestion and rescore candidates.

At least ten decided candidates with both positive and negative decisions are required.

`POST /jobs/{job_id}/resumes`

Uploads one or more resumes as multipart form data using the field name `files`.

Supported formats:

? PDF

? DOCX

? TXT

The response separates successful processing results from errors such as invalid files, oversized files, processing failures, or duplicates.

`GET /jobs/{job_id}/candidates`

Returns ranked candidates for a job.

Optional query parameters:

? `min_score`

? `status`

? `blind`

The response includes scores, fit tier, matched and missing skills, proof evidence, confirmation status, and review flags.

`GET /jobs/{job_id}/export.csv`

Exports the ranked candidate list as CSV.

Candidate operations

`GET /candidates/{candidate_id}`

Returns detailed candidate data:

? Parsed fields

? Scores and contributions

? Ranking reasons

? Proof-of-work evidence

? Integrity flags

? Feedback history

? Interview-confirmation status

Use `?blind=true` where supported to hide identity information.

`GET /candidates/{candidate_id}/resume.pdf`

Returns the original uploaded PDF when original bytes are available.

For older candidates or non-PDF uploads, the backend returns a generated PDF containing extracted text. That fallback cannot reproduce the original formatting.

`POST /candidates/{candidate_id}/feedback`

Records recruiter feedback.

Supported actions include:

? `shortlist`

? `reject`

? `interview`

? `note`

Shortlisting is required before generating an interview-confirmation link.

`POST /candidates/{candidate_id}/reverify`

Runs proof-of-work verification again for the candidate and updates the evidence and score.

`POST /candidates/{candidate_id}/confirmation-link`

Creates an expiring confirmation request for a shortlisted candidate.

Returns:

? Confirmation status

? Confirmation URL/path

? Allocated demonstration WhatsApp number

? Expiry time

`DELETE /candidates/{candidate_id}`

Deletes a candidate and related feedback/confirmation records according to configured cascade relationships.

Candidate confirmation

These endpoints use the secure token rather than recruiter JWT authentication.

`GET /whatsapp/confirm/{token}`

Returns the candidate-facing confirmation information:

? Candidate name

? Job title

? Job description

? Required skills

? Minimum experience

? Education level

? Current confirmation status

? Expiry time

`POST /whatsapp/confirm/{token}`

Records the candidate decision.

Request:

```
{
  "decision": "accepted"
}
```

Allowed values:

? `accepted`

? `declined`

The request becomes final after a decision and cannot be reused.

Database model

User

Stores recruiter account data, password hash, company, and owned jobs.

Job

Stores:

? Owner

? Title and description

? Required skills

? Minimum experience

? Education requirement

? Proof-of-work requirement

? Current scoring weights

Candidate

Stores:

? Job relationship

? Original filename

? Content hash

? Parsed name, email, and allocated demonstration phone

? Extracted raw text

? Original PDF bytes where available

? Original content type

? Parsed JSON data

? Proof-of-work evidence

? Safety flags

? Score breakdown

? Final score

? Recruiter status

The actual phone number parsed from a resume is deliberately not used for the demonstration WhatsApp allocation. The backend stores the configured allocated number for consistent recruiter confirmation behavior.

Feedback

Stores recruiter actions and notes used for auditability and optional weight learning.

WhatsAppConfirmation

Stores:

? Candidate relationship

? Hashed token

? Last four digits of allocated demonstration number

? Status

? Expiry time

? Sent time

? Response time

Security and privacy behavior

? Passwords are hashed and are never stored in plain text.

? Recruiter endpoints use JWT bearer authentication.

? Job and candidate ownership is checked on recruiter requests.

? Confirmation URLs contain random tokens; only token hashes are stored.

? Confirmation links expire.

? A confirmation decision can be submitted only once.

? Candidate deletion removes derived candidate data and related workflow records.

? Blind screening can hide identity fields during review.

? External verification is based on public information and should be disclosed appropriately.

? Secrets and tokens belong in environment variables, not source files.

? CORS is currently permissive for local development and should be restricted for production deployment.

WhatsApp demonstration behavior

The current workflow is deliberately free/manual:

? Recruiter shortlists a candidate.

? Recruiter generates a confirmation link.

? The backend assigns or reuses one configured demonstration number.

? The frontend can open WhatsApp with a prefilled message.

? The recruiter manually presses Send.

? The candidate opens the confirmation page and responds.

? The recruiter sees the response in the dashboard.

This project does not automatically send messages through the WhatsApp Cloud API. The demonstration numbers are placeholders and must not be treated as real candidate contact numbers.

Limitations and production considerations

? LeetCode verification depends on an unofficial GraphQL interface and may change.

? GitHub API limits are higher with a configured token.

? Public activity is not the same as complete developer ability.

? Skill extraction is taxonomy-based; add synonyms to `services/skills.py` when needed.

? Resume processing is synchronous and should move to a background task queue for high-volume use.

? The default frontend CORS setup is suitable for local development, not production.

? Scanned PDFs require OCR dependencies and system binaries.

? Original PDF formatting is preserved only when the original PDF bytes are available.

? DOCX and TXT files are parsed, but the PDF viewer does not convert them into their original visual format.

? Existing candidates uploaded before original-file storage cannot have their original formatting recovered.

? Database migrations should be managed with a migration tool for production deployments.

? Recruiter decisions remain essential; scores should not be used as the sole basis for employment decisions.

Recommended production improvements

? Use PostgreSQL with a migration system such as Alembic.

? Restrict CORS to the deployed frontend origin.

? Store uploaded files in private object storage rather than directly in database rows at high scale.

? Add antivirus and content-type validation before processing uploads.

? Move resume processing and external verification to a task queue.

? Add rate limiting to authentication and public confirmation endpoints.

? Add candidate consent and disclosure text for public-profile verification.

? Replace demonstration WhatsApp numbers with an approved messaging provider.

? Add audit logs for recruiter actions.

? Add automated frontend browser tests for drawer state, upload loading, and confirmation status rendering.

License and responsible use

This application is a screening aid, not an autonomous hiring decision-maker. Recruiters should verify important information directly, provide appropriate candidate disclosures, follow applicable employment law, and avoid relying on a single automated score.